The Prompt Ladder

Engineering a FastAPI prompt from weak to reusable, one change at a time

Most prompts fail quietly — they get an answer, just not a good one. This exercise takes a genuinely weak prompt and improves it in five isolated steps, one variable at a time, so it's clear exactly which change made the output better.

Baseline — Weak Prompt

PROMPT

```
Write FastAPI code.
```

OUTPUT

> A generic, single-file example with a random unrelated resource, no clear purpose, and no structure — the kind of answer that could apply to any app or none at all.

Version 1 — Add a Goal

PROMPT

```
Write FastAPI CRUD code for a Product API.
```

OUTPUT

> Code now targets an actual resource — a Product model with create, read, update, and delete endpoints — instead of arbitrary boilerplate.

Changed: Added a concrete goal.

Improved: The AI stopped guessing and built around a real resource.

Still failed: No audience in mind — explanations assumed too much or too little, and there was no sense of who the code was for.

Next: Define the audience.

Version 2 — Add Audience

PROMPT

```
Write FastAPI CRUD code for a Product API.
The explanation should be suitable for beginner FastAPI developers.
```

OUTPUT

> Explanations became simpler, with inline comments and plainer language matching someone new to FastAPI.

Changed: Added a target audience.

Improved: The explanation matched a beginner's level instead of assuming prior FastAPI knowledge.

Still failed: The code defaulted to SQLite with no ORM, which doesn't match a real production setup.

Next: Add real project context.

Version 3 — Add Context

PROMPT

```
Write FastAPI CRUD code for a Product API.
The explanation should be suitable for beginner FastAPI developers.
The project uses PostgreSQL and SQLAlchemy.
```

OUTPUT

> The code switched from SQLite to PostgreSQL with SQLAlchemy models and a proper database session setup — much closer to a real project.

Changed: Added the actual tech stack.

Improved: Generated PostgreSQL and SQLAlchemy code instead of a throwaway SQLite example.

Still failed: Everything was still dumped into one file — no separation between models, schemas, and routes.

Next: Specify an output format.

Version 4 — Add Output Format

PROMPT

```
Write FastAPI CRUD code for a Product API.
The explanation should be suitable for beginner FastAPI developers.
The project uses PostgreSQL and SQLAlchemy.

Show the answer in this order:
1. Models
2. Schemas
3. CRUD
4. API Routes
```

OUTPUT

> The response split cleanly into the four requested sections, mirroring how a real FastAPI project is actually organized into separate files.

Changed: Specified the exact structure of the response.

Improved: The code became organized and directly mappable to real project files instead of one long block.

Still failed: Used async everywhere by default and some files ran long, with no clean-code discipline.

Next: Add constraints.

Version 5 — Add Constraints

PROMPT

```
Write FastAPI CRUD code for a Product API.
The explanation should be suitable for beginner FastAPI developers.
The project uses PostgreSQL and SQLAlchemy.

Show the answer in this order:
1. Models
2. Schemas
3. CRUD
4. API Routes

Do not use async.
Keep every file under 60 lines.
Follow clean code principles.
```

OUTPUT

> Shorter, synchronous, and cleanly separated files that were easy to read end to end.

Changed: Added hard constraints on style and length.

Improved: The code got noticeably easier to read in one pass, with no bloated files.

Honest miss: The "no async" rule actually made the example less realistic — production FastAPI code almost always uses async for database calls, so this constraint traded correctness for simplicity.

Next: In real use, I'd keep the structure and clean-code rule but drop the no-async constraint.

Final Reusable Prompt

```
Create a FastAPI CRUD API for a Product Management system.

Audience:
Beginner FastAPI developers.

Context:
The project uses PostgreSQL and SQLAlchemy.

Output format:
1. Project structure
2. Models
3. Schemas
4. CRUD functions
5. API routes
6. Example requests

Constraints:
Keep code clean and beginner-friendly.
Explain important decisions briefly.
```